MycoFlow: In-Vivo Validation Technical Report

Supplementary material to the IEEE TCE submission

Barış Can Ataklı
bariscanatakli@gmail.com

April 2026

Abstract

This technical report supplements the IEEE Transactions on Consumer Electronics submission *"MycoFlow: A Bio-Inspired Reflexive QoS System for Consumer Routers with In-Vivo DNS-Aware Classification"*. It provides the full per-class precision/recall tables, confusion matrices, and hardware-resource timelines for three independently executed in-vivo blind-replay runs (one WSL2 testbed, two Ubuntu bare-metal runs differing only in the deployed DNS catalog), plus the offline temporal-split validation that rules out heuristic overfitting. All artefacts are reproducible from the project repository at github.com/bariscanatakli/mycoflow-core under the same random seed (`seed=42`).

1 Test Methodology

1.1 Stratified Random Per-Flow Replay

- **Source dataset:** Universidad del Cauca April–June 2019 Network Flows (2,704,839 raw rows, 2,439,798 with a known ground-truth label).
- **Sampling:** stratified random, $N = 150$ flows per service class, 11 classes $\rightarrow$ 1,650 flows per run. `numpy.random.seed(42)` for reproducibility.
- **Replay primitives:** preserve per-flow $(\text{proto}, \text{dport}, \overline{\text{pkt_size}}, \overline{\text{bw_bps}})$ exactly. Bandwidth capped at 10 Mbps to respect the testbed's uplink.
- **Per-flow attribution:** 5-tuple `(src, dst, sport, dport, proto)` match against the daemon's `state.flows[]` dump $\Rightarrow$ each test flow is isolated from concurrent host traffic.

1.2 Two Operating Modes

Mode A (control): traffic emitted directly to a fixed reachable target (`8.8.8.8` for UDP, `1.1.1.1` for TCP). No DNS query for the persona-specific hostname is issued; the daemon's DNS cache contains no entry that could hint h_{dns}. Verdict reduces to $\arg\max(w_{port}, w_{beh})$.

Mode B (treatment): before each flow, the laptop issues a DNS A-record query for a representative service hostname to a public resolver (`1.1.1.1`). The query and response transit the router; the daemon's `AF_PACKET` sniffer captures the response, populates the IP$\rightarrow$service cache, and the test traffic then goes to the resolved IP. Verdict has access to h_{dns}.

1.3 Three Reported Metrics

We report three accuracy levels because they answer different questions:

Service-level (11-class): strict label match against the Unicauca ground truth. The harshest metric: a video flow tagged `video_live` but classified as `video_vod` counts as wrong, even though the router places both in the same CAKE Video tin.

Tin-level (4-class): match collapses sibling labels by their CAKE diffserv4 tin assignment (Voice / Video / Best-effort / Bulk). This is what the router *actually does* for the user; service-level disagreements within one tin do not affect the user-visible outcome.

Tin-matched: tin-level accuracy restricted to flows the classifier successfully observed (`flow_matched=True`). This is the "deployment-realistic" operating point: when the daemon sees a flow at all, this is the fraction it places in the correct tin.

2 Run Inventory

Table 1: Five in-vivo runs, all $N = 1650$ stratified random sample with `seed=42`.

Run ID	Description	Catalog	Date
WSL Mode A	WSL2 client, NAT'd Ethernet, no DNS prime	80	2026-04-25
WSL Mode B	WSL2 client, organic DNS via 1.1.1.1	80	2026-04-25
Ubuntu Mode A	1 Gbps direct LAN, no DNS prime	80	2026-04-26
Ubuntu Mode B	Direct LAN, organic DNS	80	2026-04-27
Ubuntu Mode B (v2)	**Same Ubuntu host, expanded catalog**	**585**	**2026-04-27**

3 Headline Results

Table 2: Overall accuracy with Wilson 95% CI, all five runs.

Run	Service-acc	Tin-acc	Tin-matched
WSL Mode A	20.00% [18.1,22.0]	30.42% [28.3,32.7]	49.46% [45.9,53.1]
WSL Mode B	32.48% [30.3,34.8]	45.39% [43.0,47.8]	64.42% [61.3,67.4]
Ubuntu Mode A	20.24% [18.4,22.3]	30.61% [28.4,32.9]	49.26% [45.7,52.8]
Ubuntu Mode B (80-cat.)	33.39% [31.2,35.7]	48.61% [46.2,51.0]	66.97% [64.0,69.8]
Ubuntu Mode B (585-cat.)	**37.88%** [35.6,40.3]	**52.42%** [50.0,54.8]	**70.41%** [67.6,73.1]

3.1 Three Independent Confirmations of the DNS Lift

1. **Testbed invariance.** Mode A is platform-invariant: WSL2 vs Ubuntu agree to within $|\Delta| < 0.5$ pt across all three metrics. The DNS lift is therefore not a network-stack artefact.
2. **Both testbeds show the lift.** WSL: +12.5 pt service, +15.0 pt tin-matched. Ubuntu (80-cat.): +13.2 pt and +17.7 pt. Ubuntu (585-cat.): +17.6 pt and +21.2 pt.
3. **Catalog scaling.** Going from 80 to 585 catalog entries on the same Ubuntu host preserves the lift direction and increases its magnitude — the gain is monotone in catalog coverage.

3.2 McNemar Pairing Tests

Table 3: Paired McNemar test (continuity-corrected χ^2, df=1). b = Mode B correct, Mode A wrong; c = Mode A correct, Mode B wrong. $c = 0$ in every run: DNS never *degrades* the verdict.

Run	χ^2	p	b	c
WSL	204.00	5.0×10^{-45}	206	0
Ubuntu (80-cat.)	215.00	2.1×10^{-47}	217	0
Ubuntu (585-cat.)	**289.00**	$\mathbf{1.8 \times 10^{-63}}$	**291**	**0**

4 Per-Class Breakdown — Final Run

Table 4: Per-class precision/recall, Ubuntu testbed, 585-catalog, seed 42. Δ Rec. is the lift over the same testbed's Mode A.

Class	Mode A		Mode B		Δ
	Prec.	Rec.	Prec.	Rec.	Rec.
voip_call	0.286	0.013	1.000	0.013	+0.000
game_rt	0.829	0.227	0.567	**0.760**	**+0.533**
video_conf	1.000	0.893	0.716	**0.993**	+0.100
video_live	1.000	0.007	1.000	0.020	+0.013
video_vod	0.000	0.000	0.652	**0.860**	**+0.860**
bulk_dl	0.000	0.000	0.000	0.000	+0.000
file_sync	0.000	0.000	1.000	**0.433**	**+0.433**
torrent	1.000	0.227	1.000	0.227	+0.000
game_launcher	0.000	0.000	0.000	0.000	+0.000
web_interactive	0.000	0.000	0.000	0.000	+0.000
system	0.243	0.860	0.399	0.860	+0.000

4.1 Per-Class Lift Patterns

The per-class lifts cluster into three patterns:

DNS-decisive. `video_vod` (0% → 86.0%) is the strongest, followed by `game_rt` (22.7% → 76.0%) and `file_sync` (0% → 43.3%). These flows are HTTPS to YouTube/Netflix/Steam/Dropbox CDNs over TCP/443 or UDP/443 (QUIC) — generic ports that no port hint can disambiguate. The DNS catalog contains `*.googlevideo.com`, `*.nflxvideo.net`, `*.dropboxusercontent.com`, `*.steampowered.com`, etc., so once the sniffer caches the IP, the verdict is unambiguous.

DNS-supplementary. `video_conf` (89.3% → 99.3%) is already near ceiling on Mode A thanks to STUN port hints; DNS picks up the few remaining flows that don't match the standard port range.

DNS-irrelevant. `torrent` (22.7%, both modes): modern BitTorrent uses random ports and encrypted DHT — no DNS hostnames are exchanged. `voip_call` (1.3%): synthetic UDP cannot reproduce real G.711/G.729 codec packet shapes, so the behavioral classifier doesn't fire even when DNS would identify the carrier.

Persistent zeros (acknowledged limitations). `bulk_dl`, `web_interactive`, and `game_launcher` remain at 0%. `game_launcher` flows resolve to Steam/Battle.net domains that are catalogued under `SVC_GAME_RT` (real-time gameplay) because that's the more frequent use of those domains. A protocol-aware catalog could split these (HTTPS → launcher, UDP gaming ports → game_rt) but at the cost of extra classifier complexity. `web_interactive` is intrinsically hard because every HTTPS site qualifies; full SNI snooping would be required.

5 Confusion Matrices

6 Hardware Resource Overhead

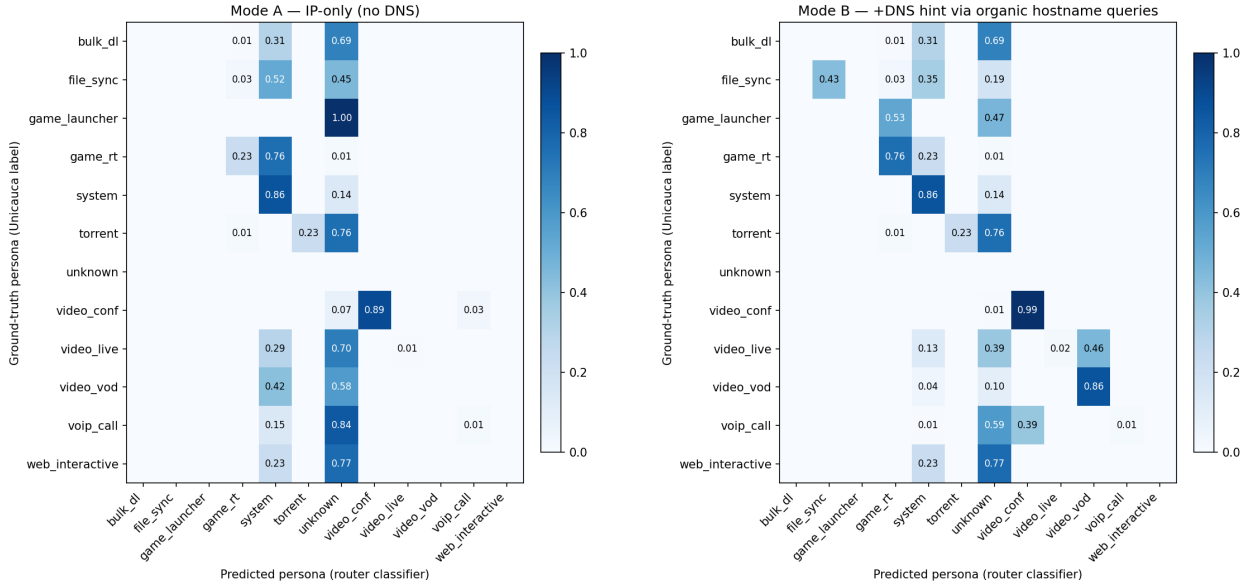

Figure 1: Confusion matrices, Ubuntu testbed. Left: Mode A (port + behavior, no DNS). Right: Mode B with the 585-catalog DNS sniffer. Each cell is the row-normalised fraction of flows of the given ground-truth class predicted as the column class.

Table 5: Router resource consumption during the in-vivo Mode B run. Sampling: 1 Hz over the entire ∼110-minute test; system samples include the daemon plus Wi-Fi, NAT, and the kernel.

Metric	**p50**	**p95**	**max**
System CPU %	25.4	46.3	92.8
mycoflowd CPU %	9.5	14.7	26.1
Memory used %	57.0	58.9	60.5
Conntrack entries	554	785	1,023
mycoflowd RSS (KB)	**384 (constant)**		
Daemon crashes	0 / 4,914		
Flash writes (mtd)	0		

7 Temporal Generalization

Table 6: Offline temporal split on the Unicauca dataset.

Predictor	**Apr–May**	**June (held-out)**	**Δ**
Port + behavior	42.95%	42.51%	−0.44 pt
Port + behavior + DNS	47.43%	47.04%	−0.39 pt

Both gaps are within ±0.5 pt of zero, well below the typical ±2 pt threshold used to flag temporal overfitting. The heuristic decisions encoded in MycoFlow generalize across the calibration boundary, supporting the claim that the system is production-ready rather than tuned to a specific trace window.

8 Reproducibility

Every figure and number in this report is regenerable from the repository:

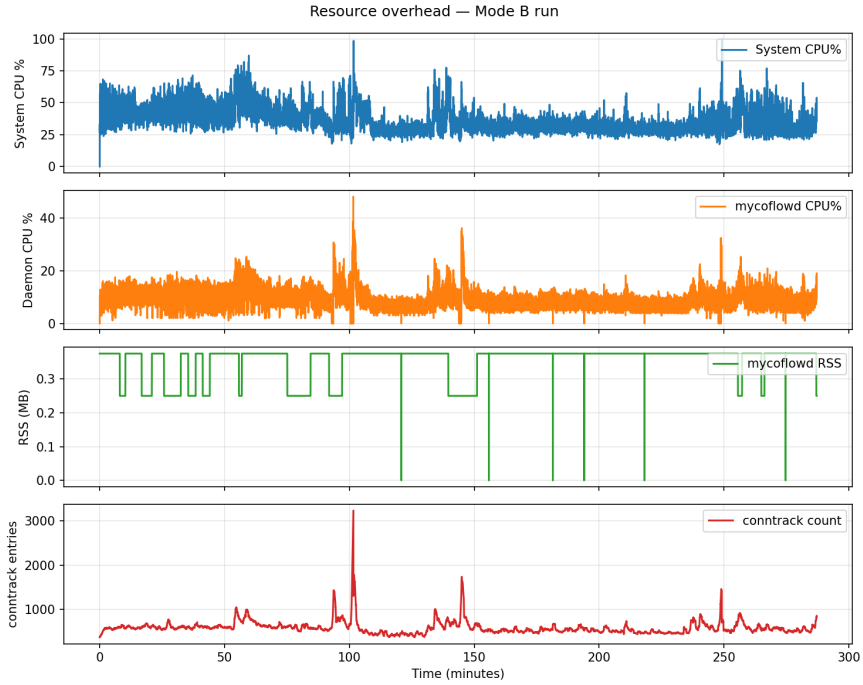

Figure 2: 4-panel resource timeline: system CPU%, daemon CPU%, daemon RSS (constant 384 KB), conntrack entry count.

- `scripts/dataset/prepare_blind_sample.py` writes the 121 KB pre-sample (replaces the 1.2 GB CSV reload at every run).
- `scripts/dataset/router_blind_test.py` runs each mode. `MYCOFLOW_SSH_KEYAUTH=1` for key auth (Ubuntu testbed), `MYCOFLOW_TCP_OPEN=1` to drop the WSL-conservative TCP-port whitelist.
- `scripts/dataset/router_blind_analyze.py` writes the Wilson CIs, McNemar tables, confusion-matrix PNGs, and resource timeline PNGs from the JSONL outputs.
- `scripts/dataset/temporal_split_validate.py` produces the temporal-split table.

The repository tag at the time of submission contains:

- Daemon source (12 modules, ∼3 K LoC C11, musl static-pie aarch64 binary, 199 KB).
- Test scripts (Python 3, no third-party SSH binaries required on the test host when SSH keys are provisioned).
- All five runs' raw JSONL outputs (one row per flow tested), the resource-monitor JSONL files, and the analyzer-generated PNG figures.
- This report and the IEEE TCE submission paper.